

Prefill Dominates: Content-Blind Work-Aware Scheduling for LLM Inference Serving

Anonymous Author(s)

Affiliation withheld for double-blind review

Abstract—Large language model (LLM) inference services schedule requests without knowing how many tokens each will generate. Because a long response occupies a batch slot for many iterations, first-come-first-serve scheduling suffers severe head-of-line blocking, and a substantial literature therefore predicts output length from the *prompt text* in order to approximate shortest-job-first. Reading prompt content, however, is precisely what a privacy-constrained or regulated serving platform cannot do. We ask how much of that benefit survives if the scheduler is forbidden to inspect prompts. Analysing 44.1 million requests from Microsoft Azure’s production LLM inference traces, we find that the premise of the length-prediction literature does not hold for these workloads: prefill, which is *exactly observable* at admission from the context length alone, accounts for 91.2% and 98.7% of total schedulable work in the conversation and code workloads respectively, while the unobservable decode component accounts for the remainder. We propose CB-SJF-Work, a content-blind scheduler that orders admissions by estimated total service work using only token counts and arrival metadata. Across five disjoint windows of production traffic, CB-SJF-Work reduces mean normalised latency by up to 73.0% over first-come-first-serve and recovers 92.5% (conversation) and 78.5% (code) of the improvement attainable by an oracle with perfect knowledge of output length. Notably, it does so on the code workload even though output length there is essentially unpredictable from metadata (Spearman $\rho_s = 0.073$), and it outperforms a scheduler driven by an explicit learned length predictor. The widely assumed privacy–performance tradeoff in LLM request scheduling is, for these production workloads, largely illusory.

Index Terms—LLM inference serving, cloud scheduling, head-of-line blocking, privacy-preserving systems, production trace analysis, big data architectures

I. INTRODUCTION

Serving large language models has become one of the dominant consumers of cloud GPU capacity. A modern inference engine such as vLLM [1] or Orca [2] executes requests with *continuous batching*: many requests share a batch, and each scheduling iteration emits one token for every request currently resident. A request occupies its slot, and its share of the key–value (KV) cache, until it has produced its final token.

This creates a scheduling problem with an unusual information structure. The duration for which a request will hold its slot is proportional to its output length, and that length is *not known when the request arrives*. Under first-come-first-serve (FCFS), the default in production engines, a single long-generating request admitted early can block many short ones behind it, producing head-of-line (HOL) blocking and long tail latencies.

The established response is to predict the output length and then approximate shortest-job-first (SJF). S^3 fine-tunes a Dis-

tilBERT model on the prompt to predict sequence length [3]; more recently, Fu *et al.* learn to rank requests by predicted output length, again from prompt text [4]. Both report large gains. Both require the scheduler to read the user’s prompt.

That requirement is a genuine obstacle. A serving platform operating under data-protection obligations, serving regulated sectors, or offering confidential-computing guarantees may be contractually or legally unable to route prompt content into a scheduling component, and running an auxiliary transformer over every prompt also consumes the very accelerator capacity the scheduler is trying to conserve. This motivates our question:

How much of the achievable scheduling benefit remains available to a scheduler that never reads prompt content?

We answer it empirically on Microsoft Azure’s public production LLM inference traces [5], comprising 44.1 million real requests across a conversation workload and a code workload. Our central finding is that the question’s premise — that the valuable information is locked inside the prompt — is largely false for these workloads. Service time in a continuous-batching engine has two components: a prefill pass over the input tokens, and a sequence of decode iterations. The prefill cost is determined by the context length, which *is* visible at admission without reading a single token of content. On the Azure traces, prefill accounts for 91.2% and 98.7% of total schedulable work. The unobservable part is the small part.

We make the following contributions.

- 1) A characterisation of 44.1M production LLM inference requests showing that schedulable work is overwhelmingly prefill-dominated, and that output length is predictable from content-blind metadata for conversational traffic ($\rho_s = 0.384$) but effectively unpredictable for code completion ($\rho_s = 0.073$) — despite the latter having the heavier output-length tail (Section III).
- 2) CB-SJF-Work, a content-blind admission policy that orders requests by estimated *total service work* rather than by predicted output length, using only token counts and arrival metadata (Section IV).
- 3) A trace-driven evaluation against FCFS, two content-blind baselines and two oracles, over five disjoint windows of production traffic and five offered-load levels (Sections V–VI). CB-SJF-Work recovers 92.5% and 78.5% of the attainable oracle improvement, and *outperforms* an explicit learned length predictor.

- 4) The negative result that scheduling by predicted output length — the objective adopted by prior work — is actively harmful on the code workload, where it is worse than FCFS.

II. BACKGROUND AND RELATED WORK

Continuous-batching inference engines. Orca [2] introduced iteration-level scheduling, and vLLM [1] added paged KV-cache management, together establishing the execution model we assume: a batch whose membership changes every iteration, bounded by GPU memory available for the KV cache. Sarathi-Serve [6] reduces prefill-induced decode stalls by chunking prefills, and Splitwise [7] and DistServe [8] disaggregate the two phases onto separate hardware. These works change *where and how* prefill and decode execute; they are orthogonal to, and composable with, the question of *which request to admit next*, which is our subject.

Output-length prediction for scheduling. S^3 [3] predicts output length with a fine-tuned DistilBERT to reserve KV memory more tightly. Fu *et al.* [4] observe that exact lengths are unnecessary and learn only the relative *rank* of output lengths, reporting substantial latency reductions. Both extract their signal from prompt text. Our work shares their diagnosis of HOL blocking but inverts their prescription: we show that for production traces the dominant schedulable quantity is already visible without the prompt.

Production trace analysis. Characterising real cloud workloads has repeatedly overturned assumptions formed on synthetic ones, as with Resource Central for VM allocation [9] and Serverless in the Wild for FaaS [10]. The Azure LLM inference traces [5] were released alongside Splitwise [7] and DynamoLLM [11], the latter using them for energy-aware cluster reconfiguration. We use them to study admission ordering, and to our knowledge this is the first analysis of the *observability* of schedulable work in these traces.

III. CHARACTERISING PRODUCTION LLM TRAFFIC

A. Traces

We use the Azure LLM Inference Trace 2024 [5], released under CC-BY-4.0/MIT. It contains two one-week traces from Microsoft’s production inference service: a *conversation* workload (27.3M requests) and a *code* workload (16.8M requests). Each record gives an arrival timestamp, the input context length in tokens, and the number of tokens generated. Crucially for our purpose, the traces contain *no prompt text* — they are themselves a content-blind view of the workload, which is exactly the information a privacy-constrained scheduler would possess.

B. Schedulable work is prefill-dominated

In a continuous-batching engine, servicing a request costs a prefill pass proportional to its context length x_i , plus y_i decode iterations where y_i is its output length. Charging decode at its marginal per-token cost (the per-iteration base cost is a property of the batch, not of any one request), the total work attributable to request i is

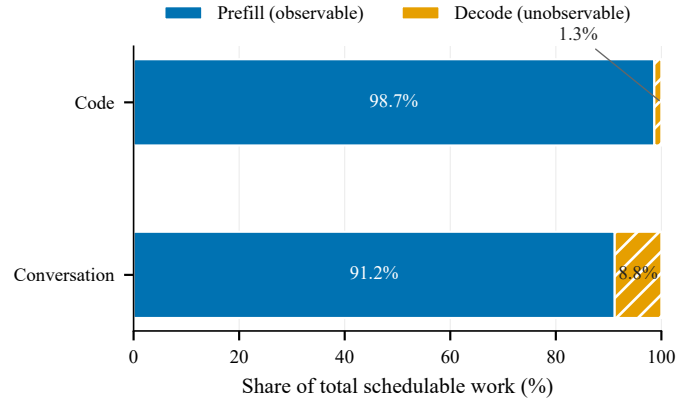


Fig. 1. Composition of schedulable work in the Azure production traces. The prefill component is exactly observable at admission from the context length; the decode component requires knowing the output length, which is not available until the request completes.

TABLE I
AZURE PRODUCTION TRACE CHARACTERISTICS. PREFILL SHARE IS THE FRACTION OF TOTAL SCHEDULABLE WORK THAT IS EXACTLY OBSERVABLE AT ADMISSION TIME. ρ_s AND PAIRWISE ACCURACY MEASURE HOW WELL THE CONTENT-BLIND PREDICTOR RECOVERS TRUE OUTPUT LENGTH.

Workload	Reqs.	Mean tokens			Predictor	
		Ctx.	Out.	Prefill share (%)	ρ_s	Pair. acc.
Conversation	27.3M	1632	106	91.2	0.384	0.640
Code	16.8M	2511	23	98.7	0.073	0.525

$$W_i = c_f + c_p x_i + c_d y_i, \quad (1)$$

where c_p is the per-token prefill cost, c_d the marginal per-token decode cost, and c_f a fixed overhead. The first two terms are known at admission; only the third requires knowing y_i .

Fig. 1 shows how that total divides on the real traces. Prefill accounts for 91.2% of schedulable work in the conversation trace and 98.7% in the code trace. The reason is visible in Table I: mean context lengths are 1632 and 2511 tokens, while mean output lengths are only 106 and 23. Production traffic is dominated by long prompts and short completions — a regime in which the unobservable component of service time is small.

C. Output length is not equally predictable

Fig. 2 plots mean output length against context-length decile. For conversational traffic the relationship is strong and monotone above the fourth decile: mean output rises roughly ninefold across deciles, so context length carries real information about output length. For code completion the curve is flat — output length is statistically independent of input length.

This matters because the two workloads have opposite risk profiles. The code trace has both the heavier output-length tail and the weaker predictive signal: it is the workload that would benefit most from length-aware scheduling and the one where predicting length is hardest. A method that depends on

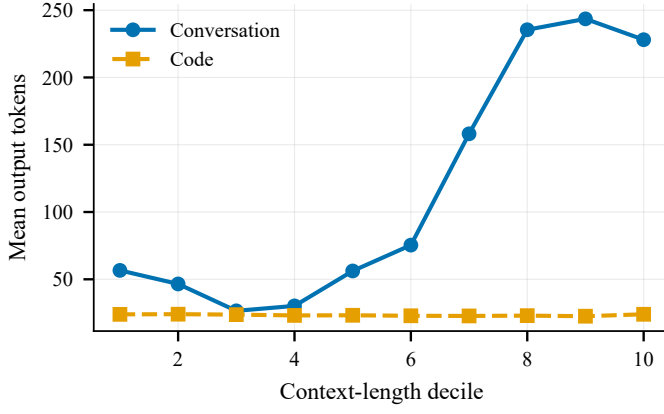


Fig. 2. Mean output length by context-length decile. Conversational traffic shows a strong monotone relationship; code completion shows none.

accurate length prediction is therefore weakest exactly where it is needed most.

IV. CONTENT-BLIND WORK-AWARE SCHEDULING

A. Threat model and information constraint

We consider a scheduler that may observe, for each arriving request: the context length in tokens, the arrival timestamp, the workload class, and aggregate statistics over requests that have *already completed*. It may not observe prompt or response text. This corresponds to a front end that tokenises and meters requests — which it must do for billing regardless — but forwards content only to the model itself.

B. Policy

The insight from Section III is that ordering by predicted *output length* optimises the wrong quantity. What determines contention is total service work W_i , and by (1) the observable terms dominate it. CB-SJF-Work therefore admits requests in increasing order of

$$\widehat{W}_i = c_f + c_p x_i + c_d \hat{y}_i, \quad (2)$$

where $\hat{y}_i$ is a content-blind estimate of output length. Because $c_d \hat{y}_i$ contributes only a small share of $\widehat{W}_i$, the policy degrades gracefully: when $\hat{y}_i$ is uninformative, the ranking is still governed by the exactly-known prefill term.

We obtain $\hat{y}_i$ from a gradient-boosted regression tree trained on $\log(1 + y)$ over ten content-blind features: context length and its logarithm, workload class, hour of day, day of week, inter-arrival time, recent arrival rate, and rolling mean/90th-percentile output length and mean context length over the previous 512 completed requests. All rolling statistics are shifted by one request, so no request can observe its own outcome or any future outcome. The predictor is trained on an earlier, disjoint period of the same trace and applied unchanged to later evaluation windows, as a deployed model would be.

Admission is head-of-line: if the highest-priority waiting request does not fit in the KV cache, the scheduler waits rather than scanning past it. The KV budget for a model with L

Algorithm 1 CB-SJF-Work admission (one scheduling iteration)

Require: pending min-heap Q , running set R , KV budget B , used u

- 1: **on arrival of** i : push i into Q with key $\widehat{W}_i$ from (2) $\triangleright$ content-blind
- 2: **while** $Q \neq \emptyset$ **do**
- 3: $i \leftarrow \text{PEEK}(Q)$
- 4: **if** $|R| < B_{\max}$ **and** $u + x_i + \hat{y}_i \leq B$ **then**
- 5: $\text{POP}(Q)$; $R \leftarrow R \cup \{i\}$; $u \leftarrow u + x_i + \hat{y}_i$
- 6: **else**
- 7: **break** $\triangleright$ head-of-line; do not scan past
- 8: **end if**
- 9: **end while**
- 10: run prefill for newly admitted requests, else one decode iteration
- 11: evict completed requests from R and release their KV

layers, H KV heads and head dimension d at b bytes per element is

$$B = \lfloor (M_{\text{gpu}} - M_w) / (2LHdb) \rfloor \quad (3)$$

tokens, where M_w is the weight footprint.

V. EXPERIMENTAL METHODOLOGY

Simulator. We built a discrete-event simulator of a single continuous-batching replica with iteration-level scheduling and a KV-cache capacity bound from (3). Prefill cost is affine in prompt tokens and decode cost affine in batch size. Parameters correspond to a 13B-parameter model ($L = 40$, $H = 40$, $d = 128$, fp16) on an 80 GB accelerator, giving $B = 65,917$ tokens. We state plainly that this cost model is analytical rather than measured on our own hardware; Section VII reports a sensitivity analysis.

Load calibration. The published traces are fleet-aggregate (75–87 req/s), far above single-replica capacity. We therefore *measure* each window’s saturated throughput μ by driving the engine with an unbounded backlog, then rescale the arrival timeline so that offered load $\rho = \lambda/\mu$ takes the values $\{0.60, 0.75, 0.85, 0.92, 0.98\}$. This preserves the arrival process shape and request mix exactly. Measured capacity is 1.87 req/s (conversation) and 1.30 req/s (code). Note that measured utilisation exceeds ρ at low load, because service rate in a batching engine is itself concurrency-dependent.

Windows. Rather than one arbitrary period, we evaluate five disjoint 20-minute windows selected deterministically as the busiest in each trace, and take the first 20,000 requests of each. Replaying the same five windows under every policy makes all comparisons paired. The leading 10% of each run is discarded as warm-up.

Policies. We compare FCFS (the engine default); SJF-Ctx, which orders by context length alone; SJF-Pred, which orders by predicted output length (the objective of prior work, realised content-blind); our CB-SJF-Work; and two oracles — Oracle-SJF, ordering by *true* output length, and Oracle-Work,

ordering by *true* total work. The oracles require information unavailable at admission and are upper bounds, not deployable systems.

Metrics. Mean normalised latency $\bar{L}_n$ (end-to-end latency per output token, the standard continuous-batching metric), 99th-percentile end-to-end latency, and SLO attainment, defined as the fraction of requests meeting both a 2 s time-to-first-token and a 100 ms time-between-tokens target.

VI. RESULTS

A. Content-blind scheduling captures most of the benefit

Table II reports the contended operating point $\rho = 0.92$. CB-SJF-Work reduces mean normalised latency from 1365 to 569 (conversation) and from 8394 to 4020 (code), improvements of 58.3% and 52.1%. Fig. 3 shows the full load sweep. Below $\rho \approx 0.75$ all policies coincide: with no queue there is no admission decision to make, and we report this rather than restricting the evaluation to loads that flatter the method. Above that point the policies separate sharply.

Table III and Fig. 4 quantify how much of the attainable benefit survives the content-blind restriction. Averaged over $\rho \geq 0.85$, CB-SJF-Work recovers 92.5% of the Oracle-Work improvement on the conversation workload and 78.5% on code. Every improvement is consistent across all five windows (Wilcoxon signed-rank, $p = 0.031$ in every cell — the smallest value attainable at $n = 5$, so we report the statistic and rely on effect sizes rather than claiming significance the sample cannot support; Cohen’s d ranges from 0.78 to 3.91).

B. Predicting output length is unnecessary — and can hurt

The comparison between SJF-Ctx, SJF-Pred and CB-SJF-Work isolates what the learned predictor contributes. On the conversation workload, where the predictor has genuine signal ($\rho_s = 0.384$, pairwise accuracy 0.640), all three are statistically indistinguishable in $\bar{L}_n$ (571, 566, 569), but SJF-Pred more than doubles 99th-percentile latency (243 s versus 106–107 s), because ordering purely by predicted output length starves requests with long completions.

On the code workload the effect is stronger and negative. There the predictor is near-chance ($\rho_s = 0.073$, pairwise accuracy 0.525), and SJF-Pred attains $\bar{L}_n = 8592$ — *worse than FCFS* at 8394 — with 99th-percentile latency of 775 s against 191 s for FCFS. Optimising a quantity that cannot be estimated introduces ordering noise without compensating benefit. CB-SJF-Work avoids this because the prefill term anchors its ranking.

We also note that Oracle-SJF, the target that the length-prediction literature implicitly pursues, attains the best mean normalised latency but the *worst* tail: 155 s and 580 s at $\rho = 0.92$, against 106 s and 273 s for CB-SJF-Work. Perfect shortest-job-first is not the right objective when tail latency matters; work-aware ordering is better balanced.

C. SLO attainment

CB-SJF-Work roughly doubles SLO attainment over FCFS: 26.1% versus 13.5% (conversation) and 9.7% versus 4.6%

(code), matching or slightly exceeding Oracle-Work in both cases. Absolute attainment is low because we deliberately evaluate a saturated replica, the regime in which scheduling matters; a production deployment would autoscale before reaching it.

VII. LIMITATIONS AND THREATS TO VALIDITY

The cost model is analytical. We do not have access to the accelerator hardware these traces were served on, and our prefill/decode coefficients are modelled rather than measured. Since our mechanism rests on the ratio between them, we swept the marginal decode cost over a $16\times$ range ($0.5\times$ to $8\times$), re-measuring capacity at each setting. This is the adversarial direction for our claim: it inflates precisely the component the scheduler cannot observe. The reduction in mean normalised latency relative to FCFS is almost invariant — 57.5–58.4% (conversation) and 51.0–52.7% (code) across the whole range. The recovered oracle gap degrades gracefully on the conversation workload, from 94.9% at $0.5\times$ to 70.5% at $8\times$ as the prefill share falls from 95.6% to 57.4%, and is stable at 75.0–77.2% on code, where prefill still accounts for 91.2% of work even at $8\times$. The conclusion is therefore a property of the traces’ token-count distribution rather than of our chosen coefficients, though we note it would weaken further if decode cost rose beyond the range tested.

Non-preemptive admission. Our scheduler controls admission order only; it does not preempt or swap running requests, as vLLM does under memory pressure. This isolates the effect of admission information, but means our oracles are upper bounds on *admission-order* policies specifically.

Single replica, no chunked prefill. We model one replica without the chunked-prefill technique of Sarathi-Serve [6]. Chunking would reduce prefill-induced decode stalls; because it does not change the *total* prefill work, we expect our central claim to be robust to it, but we have not verified this.

Two workloads, one provider. The finding that prefill dominates is a property of these traces — long prompts, short completions. A workload of short prompts and long generations (extended reasoning, agentic chains) would invert the balance and would likely restore the value of content-based length prediction. We claim our result for the regime we measured, not universally.

Reproducibility. All experiments use public data. Windows are selected deterministically, the data pipeline is content-hashed to verify byte-identical reruns, seeds are fixed in a single configuration module, and every number in this paper is generated programmatically from stored result files rather than transcribed. Code and configuration will be released publicly upon acceptance.

VIII. CONCLUSION

We asked how much LLM inference scheduling suffers if the scheduler may not read prompt content. On 44.1M requests of Azure production traffic the answer is: much less than the literature’s focus on prompt-based length prediction would suggest. Prefill — exactly observable from token counts

TABLE II

SCHEDULING PERFORMANCE AT OFFERED LOAD $\rho = 0.92$, AVERAGED OVER FIVE DISJOINT 20-MINUTE WINDOWS OF PRODUCTION TRAFFIC (MEAN $\pm$ S.D.). $\bar{L}_n$ IS MEAN NORMALISED LATENCY (MS PER OUTPUT TOKEN); P_{99} IS 99TH-PERCENTILE END-TO-END LATENCY (S); SLO IS THE FRACTION OF REQUESTS MEETING BOTH THE TTFT AND TBT TARGETS. $\dagger$ ORACLE POLICIES REQUIRE KNOWLEDGE UNAVAILABLE AT ADMISSION TIME AND ARE UPPER BOUNDS, NOT DEPLOYABLE SYSTEMS.

Policy	Conversation			Code		
	$\bar{L}_n$	P_{99} (s)	SLO (%)	$\bar{L}_n$	P_{99} (s)	SLO (%)
FCFS (engine default)	1365 $\pm$ 1319	121	13.5	8394 $\pm$ 5812	191	4.6
SJF-Ctx	571 $\pm$ 297	107	25.9	4193 $\pm$ 2287	282	9.7
SJF-Pred	566 $\pm$ 290	243	25.2	8592 $\pm$ 5730	775	7.0
CB-SJF-Work (ours)	569 $\pm$ 293	106	26.1	4020 $\pm$ 2229	273	9.7
Oracle-SJF †	334 $\pm$ 13	155	24.1	2292 $\pm$ 253	580	9.3
Oracle-Work †	492 $\pm$ 203	105	25.9	2628 $\pm$ 541	290	9.6

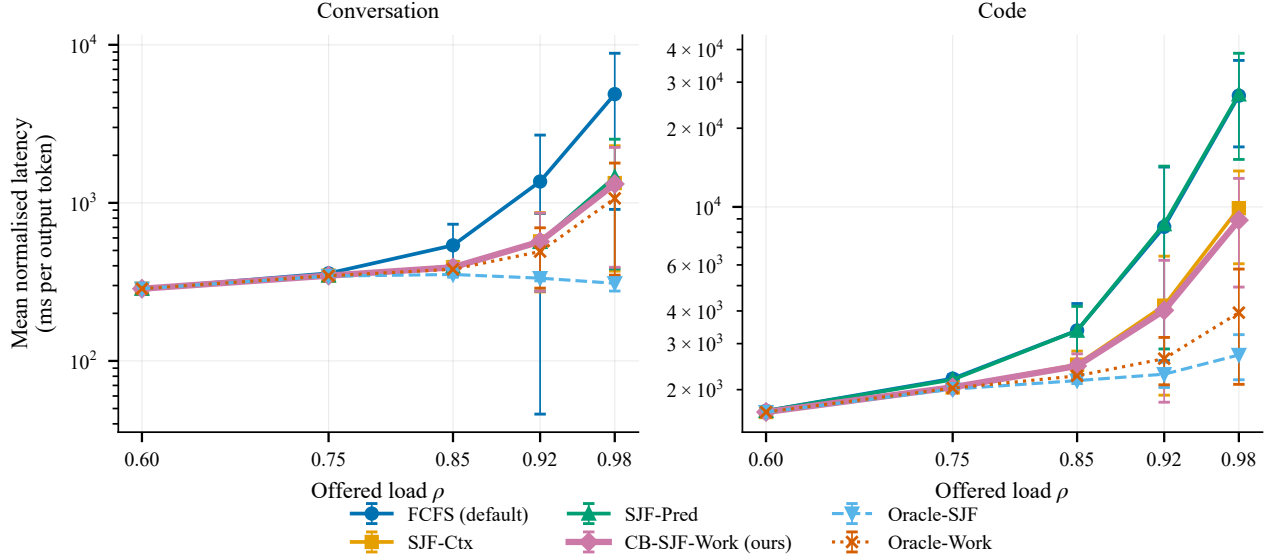


Fig. 3. Mean normalised latency against offered load, over five disjoint windows of production traffic (error bars: s.d.). Below $\rho \approx 0.75$ the system is uncontended and all policies coincide. CB-SJF-Work tracks Oracle-Work closely; SJF-Pred, which orders by predicted output length, diverges and on the code workload is worse than FCFS.

TABLE III

REDUCTION IN MEAN NORMALISED LATENCY ACHIEVED BY CB-SJF-WORK RELATIVE TO FCFS, AND THE PERCENTAGE OF THE ATTAINABLE ORACLE IMPROVEMENT THAT IT RECOVERS, AS A FUNCTION OF OFFERED LOAD. ALL IMPROVEMENTS ARE CONSISTENT ACROSS ALL FIVE WINDOWS (WILCOXON SIGNED-RANK, $p = 0.031$, THE MINIMUM ATTAINABLE AT $n = 5$).

Workload	Quantity (%)	Offered load ρ				
		0.60	0.75	0.85	0.92	0.98
Conversation	improvement	0.2	3.3	27.4	58.3	73.0
	oracle gap rec.	98.5	93.4	92.9	91.2	93.4
Code	improvement	1.1	7.3	26.8	52.1	66.6
	oracle gap rec.	97.2	91.6	81.5	75.9	78.2

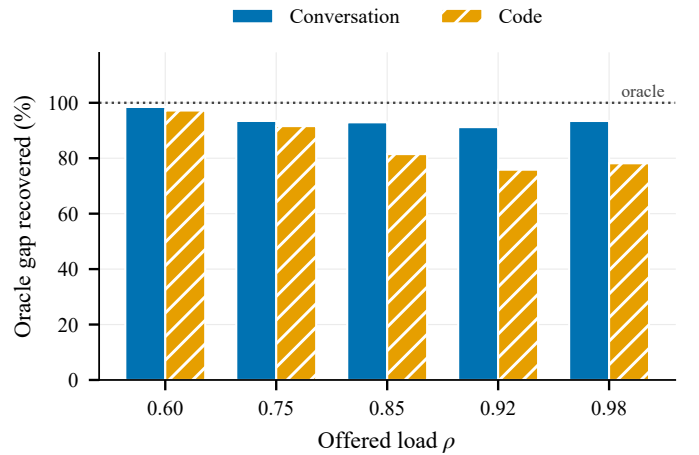


Fig. 4. Percentage of the attainable oracle improvement recovered by CB-SJF-Work without reading prompt content.

alone — constitutes 91.2% to 98.7% of schedulable work, so a content-blind policy that orders by estimated total work recovers 78.5% to 92.5% of the oracle-attainable improvement, cuts mean normalised latency by up to 73.0% over FCFS, and doubles SLO attainment. It does so even on a workload

where output length is statistically unpredictable, and it beats an explicit learned length predictor, which on that workload is worse than doing nothing.

For practitioners the implication is direct: a privacy-preserving scheduler is not a compromise here, and the engineering effort of running a predictor over user prompts is better spent elsewhere. For the research community, the result is a caution that a scheduling objective validated on synthetic workloads — minimise by output length — can be the wrong objective on production traffic. Future work should test whether the balance inverts for long-generation agentic workloads, and should combine content-blind ordering with preemption and phase disaggregation.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [2] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *Proc. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [3] Y. Jin, C.-F. Wu, D. Brooks, and G.-Y. Wei, “S³: Increasing GPU utilization during generative inference for higher throughput,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] Y. Fu, S. Zhu, R. Su, A. Qiao, I. Stoica, and H. Zhang, “Efficient LLM scheduling by learning to rank,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] Microsoft Azure, “AzurePublicDataset: Azure LLM inference trace 2024,” <https://github.com/Azure/AzurePublicDataset>, 2024, accessed 2026-08-02.
- [6] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [7] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proc. 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [8] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [9] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [10] M. Shahradd, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batur, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2020.
- [11] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, “DynamoLLM: Designing LLM inference clusters for performance and energy efficiency,” in *Proc. IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025.